



INDIAN INSTITUTE OF TECHNOLOGY
KHARAGPUR

Stamp / Signature of the Invigilator

EXAMINATION (End Semester)

SEMESTER (Spring)

Roll Number

Section

Name

Subject Number

C

S

3

0

2

0

2

Subject Name

Database Management Systems

Department / Center of the Student

Additional sheets

Important Instructions and Guidelines for Students

1. You must occupy your seat as per the Examination Schedule/Sitting Plan.
2. Do not keep mobile phones or any similar electronic gadgets with you even in the switched off mode.
3. Loose papers, class notes, books or any such materials must not be in your possession, even if they are irrelevant to the subject you are taking examination.
4. Data book, codes, graph papers, relevant standard tables/charts or any other materials are allowed only when instructed by the paper-setter.
5. Use of instrument box, pencil box and non-programmable calculator is allowed during the examination. However, exchange of these items or any other papers (including question papers) is not permitted.
6. Write on both sides of the answer script and do not tear off any page. **Use last page(s) of the answer script for rough work.** Report to the invigilator if the answer script has torn or distorted page(s).
7. It is your responsibility to ensure that you have signed the Attendance Sheet. Keep your Admit Card/Identity Card on the desk for checking by the invigilator.
8. You may leave the examination hall for wash room or for drinking water for a very short period. Record your absence from the Examination Hall in the register provided. Smoking and the consumption of any kind of beverages are strictly prohibited inside the Examination Hall.
9. Do not leave the Examination Hall without submitting your answer script to the invigilator. **In any case, you are not allowed to take away the answer script with you.** After the completion of the examination, do not leave the seat until the invigilators collect all the answer scripts.
10. During the examination, either inside or outside the Examination Hall, gathering information from any kind of sources or exchanging information with others or any such attempt will be treated as '**unfair means**'. Do not adopt unfair means and do not indulge in unseemly behavior.

Violation of any of the above instructions may lead to severe punishment.

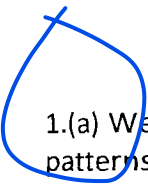
Signature of the Student

To be filled in by the examiner

Question Number	1	2	3	4	5	6	7	8	9	10	Total
Marks Obtained											
Marks obtained (in words)				Signature of the Examiner				Signature of the Scrutineer			

Instructions: Answer all SEVEN questions. Time = 3hrs. Total marks = 100. Write your answers only in the space provided. Show the solution steps. Answers without explanation will be penalised. The question paper has total 28 pages. No clarification related to questions will be provided during examination. Please state all assumptions you make.

ROUGH WORK



1.(a) We have an initially empty buffer pool with 5 buffer frames. We have six data pages U-Z. The access patterns of the pages are: X Y Z X W V U W X Y Z U V W X.

Fill up the table below for the buffer replacement policies mentioned. [3]

Policy	Number of hits	Data pages in buffer pool after completion
LRU		
MRU		
CLOCK		

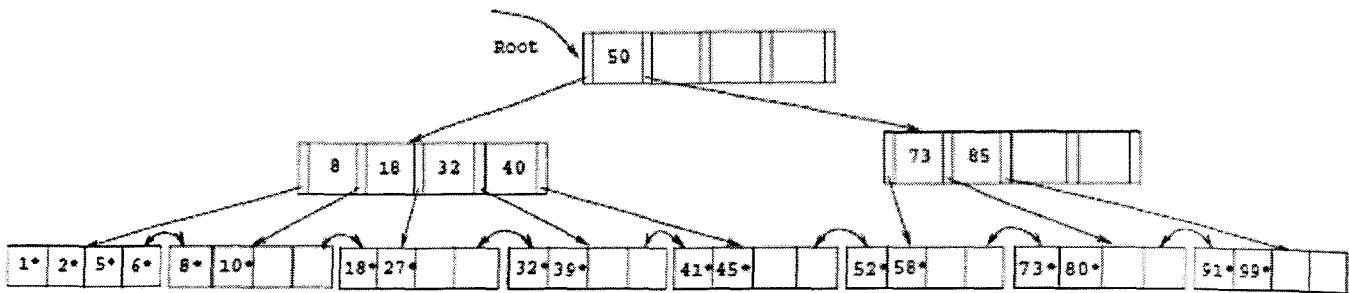
(b) Consider a disk with block size $B = 2048$ bytes. A block pointer is $P = 10$ bytes long. A file has $r = 5,000,000$ EMPLOYEE records of fixed size 400 bytes. Suppose that the file is ordered by the key field *emp_id* (15 bytes), and we want to construct a primary index on *emp_id*. Calculate -
(i) The number of first-level index entries and the number of first-level index blocks, in case of single level indexing system. Determine the number of block accesses needed to search for and retrieve a record from the file, given its *emp_id* value using the single level primary index structure. [3]

(ii) Suppose, if we want to construct an index structure on employee's Aadhar number whose size is 20 bytes (one of the attributes of the record), on the above mentioned ordered file. Assume employee's Aadhar number is NOT NULL. Determine the number of first-level index entries and the number of first-level index blocks, in case of single level indexing system. Determine the number of block accesses needed to search for and retrieve a record from the file, given its employee's Aadhar number value using the single level index structure. In case of multi-level indexing based Aadhar number, how many block accesses are required to search and retrieve a record from the file. [4]

2.(a) Suppose that we are using extendable hashing on a file that contains records with the following search-key values: 31, 77, 23, 51, 801, 163, 15, 101, 43, 82. Show the extendable hash structure incrementally (with appropriate tables and figures) for every key insertion, if the hash function is $h(x) = x \bmod 16$ and buckets can hold three records. Provide all intermediate steps and details for each key insertion. Note: Consider the start of the extendable hashing from the MSB (Most Significant Bit) value of the hash.

[10]

(b) We created the following B+ tree where each non-root node can store at most 4 keys and at most 5 pointers. When two nodes are merged the right sibling is considered first in case of a tie.



(i) Show the entire tree after inserting the key 59, and then deleting the key 91. [5]

(ii) Show the tree after sequentially deleting the keys 32, 39, 41, 45, and 73 from the original tree. [5]

3.(a) We want to sort a table which has 1960 pages with 8 available buffer pages using the external merge sort algorithm. [5]

What is the total number of passes required?	
How many sorted runs will be produced after each pass? Write comma separated values.	
How many pages will be in each sorted run for each pass?	
How many I/Os does the entire sorting operation take?	
How many buffer pages are required to sort the table in two passes?	

(b) Now we want to sort a table which has 200 pages. Suppose that during Pass 0, you have 10 buffer pages available to you, but for Pass 1 and onwards, you only have 5 buffer pages available. [5]

How many sorted runs will be produced after each pass?	
How many pages will be in each sorted run for each pass?	
How many I/Os does the entire sorting operation take?	
Does increasing the number of buffer pages does not affect the number of I/Os performed in Pass 0 of the external sort?	
Does sorting an already sorted table takes fewer IOs than sorting a randomly-arranged table?	

(c) Consider relations *Course*(*roll*, *cid*), *Hall*(*roll*, *name*, *hid*), and *Dept*(*roll*, *branch*, *bid*) to be joined on the common attribute *roll*. Assume that there are no indexes available on the tables to speed up the join algorithms. I/O cost is simply the number of pages read and written. There is an output buffer block and cost of writing the final result may be ignored.

- There are $B = 750$ pages in the buffer
- Table *Course* occupies $M = 2,500$ pages with 80 tuples per page
- Table *Hall* occupies $N = 600$ pages with 300 tuples per page
- Table *Dept* occupies $O = 1,500$ pages with 150 tuples per page
- The join result of *Course* and *Hall* occupies $P = 500$ pages

Answer the following questions. Express the answer in terms of B, M, N, O, P. [5]

What is the I/O cost of a simple nested loop join with <i>Course</i> as the outer relation and <i>Hall</i> as the inner relation?	
What is the I/O cost of a block nested loop join with <i>Hall</i> as the outer relation and <i>Dept</i> as the inner relation?	
For a sort-merge join with <i>Course</i> as the outer relation and <i>Dept</i> as the inner relation: What is the cost of sorting the tuples in <i>Course</i> on attribute <i>roll</i> ?	
For a sort-merge join with <i>Course</i> as the outer relation and <i>Dept</i> as the inner relation: What is the cost of the merge phase in the worst-case scenario?	
For a sort-merge join with <i>Course</i> as the outer relation and <i>Dept</i> as the inner relation: What is the cost of the merge phase assuming there are no duplicates in the join attribute in each table?	

4.(a) We have a schema with relation $R(BC)$, $S(ACD)$, $T(EBD)$. Show the sequence of steps (and the corresponding equivalence rule) required to transform the query $\Pi_A((R \bowtie_{B=C} S) \bowtie_{D=E} T)$ to $\Pi_A((\Pi_E(T) \bowtie_{E=D} \Pi_{ACD}(S)) \bowtie_{C=B} R)$. [3]

(b) Consider the relation $R(A, B, C)$ with the following properties:

- R has 5,000 tuples with 5 tuples per page
- Each attribute is an integer in range 1-1000
- Attribute A is a candidate key
- Unclustered hash index on attribute A
- Clustered B+ tree index on attribute B
- Attribute B has 1,000 distinct values in R
- Attribute C has 500 distinct values and an unclustered 3-level B+ tree index

What are the estimated I/O cost of the following relational algebra operations in terms of number of page access? Assume, that the best index choice is used. The indices have appropriate storage structures. [4]

Query	Cost	Index used with explanation
$\sigma_{A=90}(R)$		
$\sigma_{B=90}(R)$		
$\sigma_{C=90}(R)$		
$\Pi_{AC}(R)$		

(c) Consider the following schema:

Sailors(*sid*, *sname*, *rating*, *age*)

Reserves(*sid*, *did*, *day*)

Boats(*bid*, *bname*, *size*)

Reserves.sid is a foreign key to *Sailors* and *Reserves.bid* is a foreign key to *Boats.bid*.

We are given the following information about the database: *Reserves* contains 10,000 records with 40 records per page. *Sailors* contains 1000 records with 20 records per page. *Boats* contains 100 records with 10 records per page. There are 50 values for *Reserves.bid*. There are 10 values for *Sailors.rating* (1...10). There are 10 values for *Boat.size*. There are 500 values for *Reserves.day*

Consider the following query:

```
SELECT S.sid, S.sname, B.bname
FROM Sailors S, Reserves R, Boats B
WHERE S.sid = R.sid AND R.bid = B.bid AND
B.size > 5 AND R.day = 'July 4, 2003';
```

Assuming uniform distribution of values and column independence, estimate the number of tuples returned by this query. Explain.

5.(a) Consider the following schedule with one action missing: $r_1(A)$; $r_2(B)$; ???; $w_1(C)$; $w_2(A)$; Find out what read action could replace the ??? and make the schedule non-serializable. Explain your answer. [1]

(b) Consider the following two transactions T_1 and T_2 . How many schedules involving T_1 and T_2 are recoverable? [5]

T_1 : $w1(A)$; $w1(B)$; $r1(C)$; $commit1$;

T_2 : $w2(A)$; $r2(B)$; $w2(C)$; $commit2$;

(c) For the schedules shown below mark yes/no if they are conflict serializable, view serializable, recoverable, avoids-cascading-rollbacks. [2]

Schedule	Conflict Serializable	View Serializable	Recoverable	Avoid Cascading Roll Back
T1:R(X), T2:W(X), T1:W(X), T2:Commit, T1:Commit				
T1:R(X), T2:W(X), T1:W(X), T3:R(X), T1:Commit, T2:Commit, T3:Commit				

(d) Consider concurrency control by timestamps. Below are several sequences of events, including start events, where *sti* means that transaction *Ti* starts and *coi* means *Ti* commits. These sequences represent real time, and the timestamp-based scheduler will allocate timestamps to transactions in the order of their starts. In each case below, say what happens with the last request. You have to choose between one of the following four possible answers: accepted/ignored/rolled-back. [2]

Actions	Last action
<i>st1; st2; r2(A); co2; r1(A); w1(A);</i>	
<i>st1; st2; st3; r1(A); w2(A); w3(A); r2(A);</i>	
<i>st1; st2; r1(A); r2(A); w1(B); w2(B);</i>	
<i>st1; st2; st3; r1(A); w3(A); co3; r2(B); w2(A);</i>	

6.(a). Suppose the lock hierarchy for a database consists of database, relations, and tuples. If the database consists of hundreds of relations and millions of tuples under each relation. Determine the locks to be acquired by each of the transaction to carry out it's task. [5]

- i. Transaction T1 reads a tuple tup-7 in a relation R2.
- ii. Transaction T2 modifies a tuple tup-2 in a relation R2.
- iii. Transaction T3 wants to read all tuples of relation R2.
- iv. Transaction T4 wants to read the entire database.

In view of the above scenario, mention the various sets of transactions that can be allowed to execute concurrently?

(b). Explain, clearly (detailed steps) how a transaction T_i performs *DELETE(Q)* and *INSERT(Q)* using time stamp ordering (TSC) protocol. [4]

(c). Explain the basic principle of simple locking protocol? What are the issues in simple locking protocol in the context of concurrent execution? [2]

Explain the basic principle of two-phase locking protocol? How it addresses the problems of simple locking protocol? What are the limitations of two-phase locking protocol in view of concurrent execution? [2]

Explain the basic principle of tree protocol? How it addresses the problems faced by two-phase locking protocol? What are the limitations of tree protocol? [2]

7.(a) Consider a database with two elements, X and Y. The initial values of X and Y are both 0. We consider three transactions T_1 , T_2 , and T_3 , that modify these elements concurrently:

- $T_1: X := 42$
- $T_2: Y := 20, X := 10$
- $T_3: X := 100, Z := 101$

The log entries with some missing values (???) are shown below under different logging scenario. [10]

Pure Undo Log	Pure Redo Log	Undo-Redo Log(txn, item, before-val, after-val)
1. < START T_2 > 2. < START T_3 > 3. < $T_2, X, 0$ > 4. < $T_2, Y, 0$ > 5. < COMMIT T_2 > 6. < START CKPT(T_3) > 7. < $T_3, X, ???$ > 8. < START T_1 > 9. < $T_3, Z, 0$ > 10. < $T_1, X, ???$ >	1. < START T_2 > 2. < START T_3 > 3. < $T_2, X, 10$ > 4. < $T_2, Y, 20$ > 5. < COMMIT T_2 > 6. < START CKPT(T_3) > 7. < $T_3, X, ???$ > 8. < START T_1 > 9. < $T_3, Z, ???$ > 10. < $T_1, X, 42$ > 11. < END CKPT > 12. < COMMIT T_3 > 13. < START CKPT(T_1) > 14. < COMMIT T_1 >	1. < START T_1 > 2. < START T_2 > 3. < START T_3 > 4. < $T_3, X, 0, 100$ > 5. < $T_3, Z, 0, 101$ > 6. < COMMIT T_3 > 7. < $T_2, X, 100, 10$ > 8. < START CKPT(T_1, T_2) > 9. < $T_2, Y, ???, ???$ > 10. < END CKPT > 11. < START CKPT(T_1, T_2) > 12. < COMMIT T_2 > 13. < $T_1, X, ???, ???$ >

Pure Undo Logging	
What should be the missing entry in line 7. Explain.	
What should be the missing entry in line 10. Explain.	
If the database crashes immediately after writing the above log entries, then subsequently performs recovery. What is the state of X at the time of crash?	
Pure Redo Logging	
What should be the missing entry in line 7. Explain.	
What should be the missing entry in line 9. Explain.	
If the database crashes immediately after writing the above log entries, then subsequently performs recovery. What is the state of Y at the time of crash?	

Undo-Redo Logging	
What should be the missing entry in line 9. Explain	
What should be the missing entry in line 13. Explain	
If the database crashes immediately after writing the above log entries, then subsequently performs recovery. Which of the log entry lines will be ignored. Explain.	
Which of the log entry lines will be redone? Explain.	
Which of the log entry lines will be undone? Explain.	

(b) Consider three transactions, T_1 , T_2 and T_3 that exclusive lock (L) and unlock (U) database items X , Y and Z in the following order:

- $T_1 : L(Y) L(Z) U(Z) U(Y)$
- $T_2 : L(X) L(Z) U(X) U(Z)$
- $T_3 : L(Z) L(Y) U(Z) U(Y)$

We would like trace the execution of the transactions under the *wait-die* and *wound-wait* deadlock prevention strategies. We assume that these transactions execute in a round-robin fashion (T_1 then T_2 then T_3). When it is a transaction's turn, it executes its next lock or unlock step if it can, and otherwise dies or waits or wounds the holder of the lock, as appropriate. If a transaction is waiting when it is its turn, it skips the step and continues waiting. When a waiting transaction is restarted (due to the action of some other transaction) it becomes eligible to run at its very next turn. If a transaction A is wounded by some other transaction B , transaction B 's next action is 'Die'. Assume that the transactions have timestamps in the order of $T_1 < T_2 < T_3$ (T_1 is the oldest).

Fill in the following tables to show the sequence of steps that the three transactions make. Use $L(X)$ to denote locking an object X , $U(X)$ to denote unlocking X , "Die" to denote the transaction dying or being wounded, and "Wait" to denote the transaction waiting.

(i) *Wait-Die* Deadlock Prevention. Provide explanations.

[5]

Step	T_1	T_2	T_3
1	$L(Y)$	$L(X)$	$L(Z)$
2	Wait	Wait	Die
3			
4			
5			
6			
7			
8			
9			
10			
11			
12			

(ii) *Wound-Wait* Deadlock Prevention. Provide explanations.

[5]

Step	T_1	T_2	T_3
1	$L(Y)$	$L(X)$	$L(Z)$
2	$L(C)$	Wait	Die
3			
4			
5			
6			
7			
8			

----- END -----

ROUGH WORK

ROUGH WORK

ROUGH WORK

ROUGH WORK